Artifact Appendix

Chasing Cookies to the Source: In-browser Data Flow Backtracking for Cookie Compliance Triage

A. Description & Requirements

CookieTrace traces cookie writes back to contributing literals and scripts through in-browser data-flow backtracking. It extends Arcanum and Chromium 108.0.5359.71 with runtime maps, selective data-flow construction, forward-call tracking, and asynchronous stack stitching. The artifact contains the browser modifications, execution scripts, analysis scripts, and experimental records.

How to access. The artifact is available at https://anonymous.4open.science/r/CookieTrace. Start with ae/README.md for browser installation and execution. The repository contains measure/ for browser automation, data_processing/ for analysis, statistics/ for processed records, and ae/ for the portable runtime launcher.

Hardware dependencies. An x86-64 Linux desktop or laptop is recommended, with 8 CPU cores and 16 GB RAM and one browser worker at a time. No GPU or special hardware is required. Provide storage for the browser runtime, Docker image, experimental records, and outputs; the chrome executable alone is approximately 1.65 GB.

Software dependencies. Bash, Python >=3.9, and a local x86-64 Linux Docker Engine are used for portable execution. The Dockerfile installs Debian 12, Xvfb, and browser libraries. Use the modified Chromium 108.0.5359.71 runtime bundle. The performance aggregation scripts use Python, and the JavaScript characterization scripts use Node.js. Internet access is needed for setup and live sites; local examples and saved-record analyses do not require external websites after setup. The paper used German network egress for live measurements.

Benchmarks. The benchmarks cover consent cookies and unnecessary cookies collected from websites. The effectiveness evaluation uses an original sample set and a deduplicated subset. The statistics/ directory contains their effectiveness CSVs and processed characterization results. Source/path records provide the inputs for source-characterization analysis; instrumented and vanilla browser timing records provide the inputs for performance analysis.

B. Artifact Installation & Configuration

Place the matching browser runtime in ae/browser/sys/, so its executable is ae/browser/sys/chrome. From the repository root, run:

```
chmod u+x ae/browser/sys/chrome
bash ae/run.sh --build
bash ae/run.sh --check
bash ae/run.sh --demo
```

The build action constructs the Docker runtime; it does not compile Chromium. The checker inspects the browser bundle and runtime dependencies. Each browser run uses Xvfb, a fresh profile, and a unique output directory under ae/results/. A bind mount maps the engine's fixed internal output directory to the current run's raw/ directory. --browser-dir, --output-dir, and --proxy configure the evaluator's paths and network.

C. Experiment Workflow

Run the local examples (E1), review the supplied effectiveness tables (E2), analyze recorded runtime overhead (E3), and inspect source-characterization records (E4). measure/single_site.sh is the original per-website browser entry. ae/run.sh provides portable browser execution using --url and --cookie-name. The following evaluation steps identify the inputs and outputs of each analysis.

D. Major Claims

(C1) Source accuracy. CookieTrace identifies the source literals contributing to cookie writes. The paper reports source accuracy for the original and deduplicated evaluation sets in Tables II-III. E2 reviews the effectiveness tables used to calculate these results.

(C2) Runtime overhead. CookieTrace runtime cost is evaluated through instrumented and vanilla browser load times and analysis-stage timings. E3 aggregates the measurement records for the performance analysis in Figure 4.

(C3) Source characteristics. CookieTrace supports analysis of the scripts contributing to cookie writes and their first-party and third-party relationships. E4 examines the source-script and collaboration records associated with Tables IV-V.

E. Evaluation

Experiment (E1): Browser execution. [Preparation] Place the browser bundle in ae/browser/sys/, build the Docker runtime, and run the environment check. The readable local fixtures are in ae/docker/examples/: direct.html/direct.js test assignment from literals, and function.html/function.js test a cookie value returned by a helper function. [Execution] Run bash ae/run.sh --demo for the direct case, or add --demo-case function for the function-return case. Calling run.sh without arguments also selects the direct demo. The launcher serves the chosen fixture locally and runs the instrumented browser without an external website. [Output inspection] Open report.html in the generated run directory to view the actual cookie, source literals, script names, and positions. summary.txt provides a terminal-readable summary. Source/path JSON and logs are saved alongside the report. Expected fixture values and actual observations are shown separately.

Experiment (E2): Effectiveness (C1). The paper's effectiveness results were calculated from statistics/effectiveness_original.csv and statistics/effectiveness_deduplicated.csv. These tables contain the per-cookie ground-truth and detection counts for the original and deduplicated evaluation sets. Evaluators can inspect the supplied tables alongside the source-accuracy results in Tables II-III. This step is a review of existing records; it does not require launching CookieTrace, recollecting websites, obtaining the complete crawl, or running a calculation script.

Experiment (E3): Performance (C2). [Preparation] Select the instrumented load-time records, corresponding path JSON timestamps, and vanilla-browser timing records. [Execution] Run data_processing/summarize_overhead.py with --json_dir for the measurement root, --mode for the cookie group, --clean_file for vanilla timing records, and --output for the result JSON. The script joins records by domain and instance, applies its load-time filters, and produces per-site stage intervals, load times, and slowdown ratios. Pass the resulting JSON through standard input to data_processing/compute_avg_time.py to aggregate its numeric fields. [Output inspection] Inspect the per-site records, included samples, and aggregate timings for the Figure 4 analysis. Use recorded page-load measurements for this workflow; the launcher elapsed_seconds field describes its own execution duration.

Experiment (E4): Characterization (C3). [Preparation] Select the consent-cookie and unnecessary-cookie records in statistics/consent_cookies_statistics/ and statistics/unnecessary_cookies_statistics/. [Execution] Examine the source-script-to-domain records and the first-party, third-party, and mixed-party groupings. Preserve the website and cookie identifiers when relating these files. For analysis of path records, data_processing/characterization/ contains the corresponding JavaScript utilities. For example, summary_consent_first_and_third_party.js and summary_non-functional_first_and_third_party_domains.js select mixed-party script cases. Configure their inputDirs and outputFilePath variables to the chosen record directories and a writable output file before invoking them with Node.js. [Output inspection] Inspect the emitted domain/cookie groupings and script relationships alongside the processed records. Use consistent cookie keys, script fields, and party-classification rules when interpreting the Table IV-V analyses.